



Multi-Modal Network Intrusion Detection System Test Plan

- 1) Test Plan Identifier
- 2) References
- 3) Introduction
- 4) Test Items
- 5) Software Risk Issues
- 6) Features to be Tested
- 7) Features not to be Tested
- 8) Approach
- 9) Item Pass/Fail Criteria
- 10) Suspension Criteria and Resumption Requirements
- 11) Test Deliverables
- 12) Remaining Test Tasks
- 13) Environmental Needs
- 14) Schedule
- 15) Planning Risks and Contingencies
- 16) Glossary

1 TEST PLAN IDENTIFIER

Plan ID: NIDS-MTP-01.0

Plan Level: Master Test Plan

Software Version: 1.0 (Initial Release)

Revision: Rev 1.0 — Initial Draft

Author: QA Engineering Team

Contact: qa-team@university.edu

Date Created: 2025

Configuration Mgmt Reference: NIDS-CM-01

This document is the Master Test Plan for the Full-Stack Network Intrusion Detection System (NIDS). The plan number NIDS-MTP-01.0 identifies this as the initial master-level plan for version 1.0 of the software. Revision history will be tracked in subsequent versions of this document.

2 REFERENCES

The following documents support this test plan:

- NSL-KDD Dataset Documentation — KDD Cup 1999 dataset specification
- Full_Stack_based_Network_Intrusion_Detection.ipynb — Primary project notebook (v1.0)
- test_intrusion_detection.py — Automated test suite (v1.0)
- Train_data.csv — Training dataset, 25,192 records, 42 columns
- Test_data.csv — Test dataset
- IEEE 829-2008: Standard for Software and System Test Documentation
- scikit-learn 1.x API Documentation — <https://scikit-learn.org>
- Keras / TensorFlow 2.x Documentation — <https://keras.io>
- Qiskit Machine Learning Documentation — <https://qiskit.org/ecosystem/machine-learning>

3 INTRODUCTION

This document is the Master Test Plan for the Full-Stack Network Intrusion Detection System (NIDS). The system classifies network traffic as either normal or anomalous using the NSL-KDD dataset (41 features + binary class label). It integrates Classical Machine Learning (KNN, Decision Tree, Random Forest, SVM, Naive Bayes, XGBoost), Deep Learning (DNN), and Quantum Machine Learning (QSVM, VQC) classifiers into a comparative evaluation framework.

The primary purpose of this plan is to define the scope, approach, resources, and schedule for all testing activities. Testing is organized at three levels: Unit, Integration, and System/Acceptance. The automated test suite (test_intrusion_detection.py) implements 47 test cases covering data loading, preprocessing, model training, metrics, serialization, and edge cases.

The scope of this plan covers the complete ML pipeline from raw CSV ingestion through model persistence. Quantum ML modules are tested in simulation only. The Streamlit web interface and any cloud deployment are outside the scope of this version of the plan.

4 TEST ITEMS (FUNCTIONS)

The following components are in scope for testing under this plan:

- **A. Training Dataset (Train_data.csv) — v1.1**
 - 25,192 records, 42 columns (41 features + class label)
 - Binary target: 'normal' and 'anomaly'
 - Categorical features: protocol_type, service, flag
- **B. Test Dataset (Test_data.csv) — v1.1**
 - Held-out evaluation set with same schema as Train_data.csv
- **C. Data Preprocessing Pipeline — v1.1**
 - LabelEncoder for categorical columns
 - StandardScaler for feature normalization
 - MinMaxScaler and PCA (4 components) for Quantum ML sub-pipeline
 - 80/20 stratified train-validation split
- **D. Classical ML Models — v1.1**
 - K-Nearest Neighbors (KNeighborsClassifier)
 - Decision Tree (DecisionTreeClassifier)
 - Random Forest (RandomForestClassifier)
 - Support Vector Machine (SVC with probability=True)
 - Naive Bayes (GaussianNB)
 - XGBoost (XGBClassifier)
- **E. Deep Neural Network (DNN) — v1.1**
 - Architecture: Input(41) → Dense(64,relu) → Dropout(0.2) → Dense(32,relu) → Dense(1,sigmoid)
 - Optimizer: Adam | Loss: Binary Cross-Entropy | Format: .keras
- **F. Quantum ML Models (Simulation) — v1.1**
 - Quantum SVM (QSVC) with ZZFeatureMap, reps=2
 - Variational Quantum Classifier (VQC) with RealAmplitudes ansatz and COBYLA optimizer
- **G. Model Comparison & Metrics Module — v1.1**
 - Accuracy, Precision, Recall, F1-Score (weighted avg)
 - Sorted DataFrame output and CSV export (model_comparison_results.csv)
- **H. Model Serialization — v1.1**
 - joblib: scaler.pkl, pca.pkl, encoders.pkl, best_ml_model.pkl, accuracies.pkl
 - dnn_model.keras (native Keras format)

5 SOFTWARE RISK ISSUES

The following risk areas have been identified for this project:

- **A. Quantum ML Simulation Overhead (HIGH)**
 - QSVC and VQC run on classical simulators; training on even 100 samples takes significant time.
 - Risk: Tests may time out in CI/CD pipelines. Mitigation: Quantum tests are marked with pytest.importorskip and limited to 100 training/50 validation samples.
- **B. Third-Party Library Dependencies (HIGH)**
 - qiskit, qiskit-machine-learning, qiskit-algorithms, xgboost, tensorflow/keras require specific versions.

- Risk: API changes or incompatibilities between library versions can break the pipeline.
- **C. Data Leakage Risk (MEDIUM)**
 - Scalers and encoders must be fitted only on training data and applied to validation/test data.
 - Risk: Fitting on the full dataset would inflate accuracy metrics.
- **D. Unknown Labels in Test Set (MEDIUM)**
 - The test CSV may contain service/protocol values not seen during training.
 - Mitigation: The notebook's lambda-based mapping returns -1 for unseen labels. Verified in TestPreprocessing.test_unknown_service_handling.
- **E. Class Imbalance (MEDIUM)**
 - NSL-KDD may have imbalanced class distribution, which can bias accuracy metrics.
 - Mitigation: Weighted-average Precision, Recall, and F1-Score are collected alongside accuracy.
- **F. DNN Non-Determinism (LOW)**
 - Neural network training involves randomness. Results may vary across runs unless seeds are fixed.
 - Mitigation: DNN tests use a single epoch and check output shape/bounds rather than specific accuracy.

6 FEATURES TO BE TESTED

The following user-visible features and system behaviors are in scope:

Feature / Function	Description	Risk
Data Ingestion	Load Train/Test CSV, validate schema, check for nulls, verify column count (42)	HIGH
Categorical Encoding	Transform protocol_type, service, flag to integer labels; handle unseen categories	HIGH
Feature Scaling (Standard)	Fit StandardScaler on train set; transform train and validation sets	HIGH
Train/Validation Split	80/20 split with random_state=42; no overlap between splits	MEDIUM
Classical ML Training	Train KNN, DT, RF, SVM, NB, XGBoost; predict on validation set	HIGH
Binary Classification Output	All model predictions must be 0 (normal) or 1 (anomaly) only	HIGH
Probability Estimation	RF and SVC (probability=True) must return probabilities summing to 1.0 per row	MEDIUM
DNN Architecture & Training	Build 3-Dense-layer DNN; compile with Adam/BCE; fit and produce sigmoid output	MEDIUM

Quantum Preprocessing (PCA)	MinMaxScaler [0,1] + PCA to 4 components for QSVC/VQC input	MEDIUM
Model Metrics Collection	Compute Accuracy, Precision, Recall, F1-Score; produce sorted comparison DataFrame	MEDIUM
Model Persistence	Serialize/deserialize scaler, encoders, ML model, DNN, and accuracies dict via joblib	HIGH
Robustness / Edge Cases	All-normal input, all-anomaly input, single-row inference, extreme byte values, zero-variance features	MEDIUM
Prediction Speed	RandomForest must classify 100 samples in under 2 seconds	LOW

7 FEATURES NOT TO BE TESTED

The following areas are explicitly excluded from this test plan:

- **A. Streamlit Web Application UI**
 - Reason: Front-end rendering and user interaction testing require a separate UI test plan and tooling (e.g., Selenium). Not included in this release scope.
- **B. Quantum Hardware Execution**
 - Reason: Quantum models are tested in simulation only (Qiskit statevector simulator). Real quantum hardware access requires specialized infrastructure.
- **C. XGBoost Model (Automated Test Suite)**
 - Reason: XGBoost is trained in the notebook but the automated pytest suite does not parameterize it due to dependency isolation. It is covered by manual notebook execution.
- **D. Network-Level Security of the Application**
 - Reason: This plan covers ML pipeline correctness, not deployment security. Penetration testing and API security are out of scope.
- **E. Cloud Deployment / Containerization**
 - Reason: Docker images, cloud endpoints, and infrastructure are not part of this release. A separate DevOps test plan is required.
- **F. Pyngrok / Tunnel Stability**
 - Reason: ngrok tunneling for the Streamlit app is an infrastructure concern, not an ML pipeline concern. Not testable in this context.

8 APPROACH (STRATEGY)

8.1 Testing Levels

- **Unit Testing:** Each function (preprocessing, model training, metric computation) is tested in isolation using pytest fixtures with in-memory data. No CSV file I/O is required for unit tests.
- **Integration Testing:** The full pipeline from CSV loading through model evaluation is exercised end-to-end in a dedicated integration test class using the actual Train_data.csv and Test_data.csv files.
- **System / Acceptance Testing:** The complete notebook is executed top-to-bottom. All six classical models, the DNN, and both quantum models must complete without error and produce a sorted comparison table.

8.2 Test Tooling

- pytest — primary test runner (pytest test_intrusion_detection.py -v)
- pytest.mark.parametrize — used to run identical assertions across all classical model classes
- pytest.importorskip('keras') — gracefully skips DNN tests if TensorFlow/Keras is unavailable
- pytest fixtures (raw_train_sample, preprocessed_data) — provide deterministic, reproducible in-memory datasets
- numpy.testing — used for array-level assertions (assert_array_equal, assert_allclose)
- joblib — tested for round-trip serialization of all pipeline artifacts

8.3 Metrics Collected

- Test pass rate (passed / total)
- Test execution time per class (reported by pytest)
- Model accuracy, precision, recall, F1-score (weighted average) per classifier
- Best model selection result (highest accuracy classifier name)

8.4 Configuration Management

- All test files are version-controlled alongside the notebook
- Library versions are pinned via requirements.txt or pip install commands in the notebook header
- Datasets (Train_data.csv, Test_data.csv) are treated as fixed configuration items at v1.0

8.5 Regression Testing

- The full pytest suite is re-run on every change to the preprocessing pipeline or any model's hyperparameters
- DNN tests are run only when TensorFlow is available in the environment
- Quantum tests run in simulation mode only and are not gated for production deployments

8.6 Meetings

The test team will review test results after each pipeline change. A biweekly sync with the development lead will track defect trends and milestone progress.

9 ITEM PASS/FAIL CRITERIA

9.1 Test Case Level

- PASS: The test function executes without raising any AssertionError or unhandled exception.
- FAIL: Any AssertionError or unexpected exception is raised, or the test times out.

9.2 Test Class Level

- All Data Loading tests (8 cases) must PASS before integration tests are run.
- All Preprocessing tests (8 cases) must PASS before model training tests are run.
- At least 10 of 11 Classical Model tests must PASS for the pipeline to be considered stable.
- DNN tests are informational if Keras is unavailable; skipped tests do not count as failures.
- All Serialization tests (4 cases) must PASS before a release artifact is produced.

9.3 System Level

- The complete notebook must execute end-to-end without any Python exception.

- At least 5 of 6 classical models must achieve accuracy $\geq 85\%$ on the validation set.
- Random Forest must achieve accuracy $\geq 85\%$ (explicitly asserted in `test_random_forest_accuracy_above_threshold`).
- The sorted comparison DataFrame must list models in descending accuracy order.
- All serialized artifacts (`scaler.pkl`, `encoders.pkl`, `best_ml_model.pkl`, `dnn_model.keras`) must be loadable and produce identical output to their in-memory counterparts.

9.4 Defect Severity

Severity	Definition	Action Required
Critical	Pipeline crashes or produces NaN/Inf predictions	Block release; fix immediately
Major	Model accuracy below 85% or serialization fails	Fix before acceptance sign-off
Minor	Single model slightly underperforms; non-blocking	Log and monitor; fix in next sprint

10 SUSPENSION CRITERIA AND RESUMPTION REQUIREMENTS

- **A. Data Loading Failure**
 - If either `Train_data.csv` or `Test_data.csv` cannot be loaded (file missing, schema mismatch, >0 null values), all downstream tests are suspended immediately.
 - Resumption: Correct the dataset file and re-run the Data Loading test class. All 8 tests must pass.
- **B. Critical Preprocessing Defect**
 - If label encoding or scaling produces NaN or Inf values, model training tests are suspended.
 - Resumption: Fix the preprocessing function and re-run `TestPreprocessing`. All 8 tests must pass.
- **C. Quantum Library Unavailability**
 - If `qiskit` or `qiskit-machine-learning` cannot be installed, Quantum ML tests are suspended (not failed).
 - Resumption: Install the required packages and re-run. Classical and DNN tests continue unaffected.
- **D. More Than 5 Test Failures in a Single Run**
 - If more than 5 test cases fail in a single pytest invocation, testing is paused to investigate root cause before continuing.
 - Resumption: Identify and fix the root cause. Re-run the full suite. Must achieve fewer than 3 failures.

11 TEST DELIVERABLES

- This Master Test Plan document (NIDS-MTP-01.0.docx)
- Automated test suite: `test_intrusion_detection.py` (47 test cases, 7 test classes)

- pytest execution log / HTML report (pytest --html=report.html)
- Model comparison CSV: model_comparison_results.csv
- Serialized model artifacts: scaler.pkl, pca.pkl, encoders.pkl, best_ml_model.pkl, accuracies.pkl, dnn_model.keras
- Defect / incident log (tracked in project issue tracker)

12 REMAINING TEST TASKS

Task	Assigned To	Status
Develop XGBoost pytest test cases	Tester	Completed
Develop Streamlit UI test plan	QA Lead	Not Started
Develop quantum hardware integration tests	Quantum ML Engineer	Not Started
Establish CI/CD pipeline for automated test runs	DevOps	Not Started
Write acceptance test plan (end-user validation)	QA Lead, PM	Not Started
Performance benchmarking across full 25K dataset	Tester	Completed

13 ENVIRONMENTAL NEEDS

- Python 3.9+ runtime environment with the following packages:
 - pandas, numpy, scikit-learn, xgboost, tensorflow/keras, qiskit, qiskit-machine-learning, qiskit-algorithms, joblib, dill, matplotlib, seaborn, pytest
- Dataset files (fixed configuration items):
 - Train_data.csv — 25,192 rows, 42 columns, placed in the working directory
 - Test_data.csv — same schema, placed in the working directory
- Hardware:
 - Minimum: 8 GB RAM (Quantum simulation requires additional memory for statevector)
 - Recommended: 16 GB RAM, multi-core CPU for RandomForest and SVM training
- No network connectivity required for unit and integration tests (all data is local).
- pytest version ≥ 7.0 is required for full parametrize and importorskip support.
- tmp_path pytest fixture requires pytest ≥ 3.9 for serialization tests.

14 SCHEDULE

Activity	Relative Date	Dependency
Dataset validation and schema tests	Day 1	Datasets delivered
Preprocessing pipeline tests	Day 1–2	Schema tests passed
Classical ML model tests	Day 2–3	Preprocessing tests passed
DNN model tests	Day 3	Keras available in environment

Metrics and serialization tests	Day 3–4	Model tests passed
Edge case and robustness tests	Day 4	Core pipeline stable
Full notebook end-to-end run	Day 4–5	All unit/integration tests passed
Test report and sign-off	Day 5	All acceptance criteria met

All dates are relative to the start of the test phase. If any upstream development activity is delayed, the corresponding test activity shifts by the same number of days. Testing can be compressed to recover schedule

15 PLANNING RISKS AND CONTINGENCIES

- **Quantum Library Breakage**
 - Risk: Qiskit or qiskit-machine-learning API changes break QSVC/VQC tests.
 - Contingency: Pin library versions in requirements.txt. Quantum tests are skipped (not failed) when libraries are unavailable.
- **Dataset Unavailability**
 - Risk: Train_data.csv or Test_data.csv is not present in the expected directory.
 - Contingency: The in-memory raw_train_sample fixture allows unit tests to run without CSV files. Data loading tests will fail and block integration tests only.
- **Environment Setup Delays**
 - Risk: Installing TensorFlow, Qiskit, and XGBoost together can produce dependency conflicts.
 - Contingency: Use a pre-built Docker image with all dependencies pinned. Provide a requirements.txt with tested version combinations.
- **Model Non-Determinism**
 - Risk: Random Forest and DNN results may differ across environments due to seed differences.
 - Contingency: Tests assert structural properties (output shape, label set, probability bounds) rather than exact accuracy values. Only Random Forest has an accuracy threshold test ($\geq 85\%$), which is robust across seeds.
- **CI/CD Pipeline Not Available at Launch**
 - Risk: Automated test execution cannot be integrated into a CI pipeline before the first release.
 - Contingency: Manual pytest execution by QA Engineer is required. Log results manually in the test summary report. CI/CD setup is a remaining test task.

16 GLOSSARY

Term / Acronym	Definition
NIDS	Network Intrusion Detection System
NSL-KDD	Improved version of the KDD Cup 1999 dataset used for network intrusion detection research
KNN	K-Nearest Neighbors — a classification algorithm based on proximity in feature space
DT	Decision Tree — a tree-based classification algorithm
RF	Random Forest — an ensemble of Decision Trees using bagging
SVM / SVC	Support Vector Machine / Classifier — finds an optimal hyperplane separating classes
NB	Naive Bayes — probabilistic classifier based on Bayes theorem with feature independence assumption
XGBoost	Extreme Gradient Boosting — a gradient-boosted ensemble classifier
DNN	Deep Neural Network — a multi-layer feedforward neural network trained with backpropagation
QSVC / QSVM	Quantum Support Vector Classifier — SVM using a quantum kernel computed via a feature map circuit
VQC	Variational Quantum Classifier — a parameterized quantum circuit trained via a classical optimizer (COBYLA)
PCA	Principal Component Analysis — dimensionality reduction technique used to reduce 41 features to 4 for quantum models
BCE	Binary Cross-Entropy — loss function used for binary classification in the DNN
F1-Score	Harmonic mean of Precision and Recall; used as the primary metric for imbalanced classification
joblib	Python serialization library used to save and load scikit-learn models and scalers as .pkl files
pytest	Python testing framework used to organize and execute all automated test cases
fixture	In pytest, a reusable setup function that provides test data or objects to test functions
DXA	Device-independent eXtended Attribute — unit of measurement used in Word documents; 1440 DXA = 1 inch